



République du Sénégal
Ministère de l'enseignement supérieur de la Recherche
et de l'Innovation (MESRI)



Université Numérique Cheikh Hamidou Kane (ex UVS)

Pôle : Science Technologie et Numérique (STN)

Formation : Master en Informatique Conception &
Développement d'Application Full-Stack

Projet Semestre 2

Module : Web Services

Promotion : 8

Projet Portant Sur :

**Plateforme de Gestion des Jeux Olympiques
(Olympic Management System)**

Tutrice : Mme Seck

Mes Informations :

Prénom : El Hadji Sarr

Nom : **Diallo**

E-mail institutionnel : elhadjsarr.diallo@unchk.edu.sn

INE : **N04468520201**

Application de gestion des Jeux olympiques

**Java 21 • Spring Boot 3 • MySQL •
REST • SOAP**

Plan

- 1.Contexte et objectifs du projet**
- 2.Architecture générale de la solution**
- 3.Modèle de données**
- 4.Fonctionnalités et règles de gestion**
- 5.API REST et documentation Swagger**
- 6.Service SOAP**
- 7.Tests, démonstration et résultats**
- 8.Difficultés rencontrées et solutions**
- 9. Conclusion et perspectives**

Ce document présente la conception, l'implémentation et la validation de l'application Olympic Management System. Les zones grises prévues pour les captures peuvent être remplacées par les captures réelles de Swagger, Postman SoapUI et MySQL avant la soutenance.,

1. Contexte Et Objectifs Du Projet

Le projet consiste à développer une application backend capable de centraliser les principales informations d'une compétition olympique. L'application doit assurer la gestion des disciplines, des athlètes, des épreuves et des résultats. Elle produit également un tableau des médailles et des indicateurs statistiques destinés au suivi global de la compétition.

Objectifs fonctionnels

- *Créer, consulter, modifier et supprimer les disciplines sportives.*
- *Gérer les athlètes et leur rattachement à une discipline.*
- *Programmer les épreuves et les associer à une discipline.*
- *Enregistrer les performances et attribuer automatiquement les médailles.*
- *Calculer le tableau des médailles par nationalité.*
- *Fournir un tableau de bord statistique.*
- *Exposer les fonctionnalités à travers des API REST et un service SOAP.*

Objectifs techniques

- Mettre en œuvre une architecture en couches claire et maintenable.
- Utiliser Spring Data JPA pour la persistance MySQL.
- Documenter les API REST avec OpenAPI/Swagger.
- Mettre en œuvre un service SOAP contract-first à partir d'un schéma XSD.
- Gérer les validations, les doublons et les erreurs métier de manière uniforme.

2. Architecture Générale De La Solution

L'application adopte une architecture en couches. Chaque couche possède une responsabilité précise, ce qui facilite les tests, la maintenance et l'évolution du projet.

Couche	Rôle	Exemples
Controller	Expose les endpoints HTTP REST.	AthleteController, DashboardController
SOAP Endpoint	Traite les messages SOAP.	TableauMedailleEndpoint
Service	Porte la logique métier.	ResultatServiceImpl, DashboardServiceImpl
Repository	Accède aux données via JPA.	AthleteRepository, ResultatRepository
Mapper / DTO	Sépare les entités de l'API.	AthleteMapper, DashboardResponse
Entity	Modélise les tables de la base.	Athlete, Discipline, Epreuve, Resultat
Exception	Uniformise les réponses d'erreur.	GlobalExceptionHandler

➤ Flux d'une requête REST

Client → **Controller** → **Service** → **Repository** → **MySQL**. La réponse suit le chemin inverse après conversion des entités en DTO. Cette séparation empêche l'exposition directe des entités JPA.

➤ Choix techniques

- ☞ **Java 21** pour bénéficier d'une version moderne et stable du langage.
- ☞ **Spring Boot 3.5** pour l'auto-configuration et la productivité.
- ☞ **Maven** pour la gestion des dépendances et la génération des classes JAXB.
- ☞ **MySQL** pour la persistance relationnelle.
- ☞ **SpringDoc OpenAPI** pour la documentation interactive.
- ☞ **Spring Web Services** pour l'exposition SOAP.

Architecture du Projet

Project ▾



▾ **OlympicManagementSystem** C:\OlympicManagement\OlympicManagementSystem

> .idea

▾ .mvn

▾ wrapper

maven-wrapper.properties

▾ src

▾ main

▾ java

▾ com.olympics.management

> config

> controller

> dto

> entity

> enums

> exception

> mapper

> repository

> service

> soap

OlympicManagementSystemApplication

org.example.com.olympics

▾ resources

static

templates

> xsd

application.properties

> test

> target

.gitattributes

.gitignore

mvnw

mvnw.cmd

Olympic_Management_System.postman_collection.json

pom.xml

README.md

> External Libraries

Scratches and Consoles

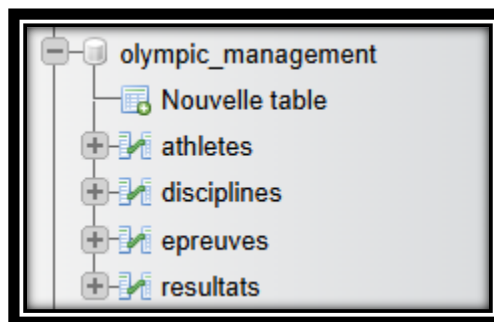
3. Modèle De Données

Le modèle repose sur quatre entités principales. Une discipline possède plusieurs athlètes et plusieurs épreuves. Un résultat associe un athlète à une épreuve avec un classement, une performance et une médaille.

Entité	Attributs clés	Relations	Contraintes principales
Discipline	id, nom, description	1-N Athlete, 1-N Epreuve	Nom obligatoire
Athlete	id, nom, prénom, nationalité, email	N-1 Discipline	Email unique
Epreuve	id, nom, date, lieu	N-1 Discipline	Discipline obligatoire
Resultat	id, classement, performance, médaille	N-1 Athlete, N-1 Epreuve	Couple athlete/epreuve unique

➤ Règles d'intégrité

- Un athlète ne peut avoir qu'un seul résultat pour une même épreuve.
- La discipline de l'athlète doit correspondre à celle de l'épreuve.
- Le classement doit permettre le calcul automatique de la médaille.
- Une adresse email ne peut être utilisée que par un seul athlète.



➤ Jeu de données de démonstration

Le fichier data.sql fourni initialise trois disciplines, neuf athlètes, trois épreuves et neuf résultats. Il permet de tester immédiatement les endpoints et d'obtenir un tableau des médailles non vide. Les insertions respectent l'ordre des clés étrangères.

4. Fonctionnalités Et Règles De Gestion

➤ Gestion CRUD

Les modules **Discipline**, **Athlete**, **Epreuve** et **Resultat** disposent chacun d'opérations de création, consultation, modification et suppression. Les contrôleurs **REST** utilisent des **DTO** de requête et de réponse afin de limiter le couplage avec le modèle de persistance.

➤ Attribution automatique des médailles

Classement	Médaille calculée
1	OR
2	ARGENT
3	BRONZE
4 ou plus	AUCUNE

➤ Tableau des médailles

Le tableau regroupe les résultats par nationalité et compte les médailles d'or, d'argent et de bronze. Le classement officiel est trié par nombre de médailles d'or, puis d'argent, puis de bronze.

➤ Dashboard

- *Nombre total d'athlètes.*
- *Nombre de pays participants.*
- *Total des médailles par type.*
- *Classement pondéré : or = 7 points, argent = 4 points, bronze = 1 point.*
- *Nombre d'athlètes médaillés par pays.*
- *Résumé complet dans une seule réponse JSON.*

➤ Gestion des erreurs

Le **GlobalExceptionHandler** transforme les exceptions en réponses JSON cohérentes. Les ressources introuvables renvoient une erreur 404, les doublons et violations métier une erreur **400**, et les erreurs de validation détaillent les champs incorrects.

5. API REST et documentation Swagger

Swagger est disponible à l'adresse <http://localhost:8081/swagger-ui/index.html>. Il permet de consulter tous les endpoints, les paramètres attendus, les schémas JSON et d'exécuter directement les requêtes.

Module	Route principale	Méthodes
Disciplines	/api/disciplines	GET, POST, PUT, DELETE
Athlètes	/api/athletes	GET, POST, PUT, DELETE
Épreuves	/api/epreuves	GET, POST, PUT, DELETE
Résultats	/api/resultats	GET, POST, PUT, DELETE
Tableau des médailles	/api/tableau-medailles	GET
Dashboard	/api/dashboard	GET

Quelques captures dans le Swagger :

Tableau de bord Statistiques générales des Jeux olympiques		
GET	/api/dashboard/summary	Tableau de bord complet
GET	/api/dashboard/medals/count	Nombre de médailles
GET	/api/dashboard/medalists-by-country	Nombre de médaillés par pays
GET	/api/dashboard/countries/ranking	Classement des pays par points
GET	/api/dashboard/countries/count	Nombre de pays participants
GET	/api/dashboard/athletes/count	Nombre total d'athlètes
resultat-controller		
GET	/api/resultats/{id}	
PUT	/api/resultats/{id}	
DELETE	/api/resultats/{id}	
GET	/api/resultats	
POST	/api/resultats	
GET	/api/resultats/epreuve/{epreuveId}	
GET	/api/resultats/athlete/{athleteId}	

epreuve-controller

GET /api/epreuves/{id}

PUT /api/epreuves/{id}

DELETE /api/epreuves/{id}

GET /api/epreuves

POST /api/epreuves

GET /api/epreuves/discipline/{disciplineId}

discipline-controller

GET /api/disciplines/{id}

PUT /api/disciplines/{id}

DELETE /api/disciplines/{id}

GET /api/disciplines

POST /api/disciplines

athlete-controller

GET /api/athletes/{id}

PUT /api/athletes/{id}

DELETE /api/athletes/{id}

GET /api/athletes

POST /api/athletes

tableau-medaille-controller

GET /api/tableau-medailles

home-controller

GET /

Schemas		
ResultatRequest >	Expand all	object
ResultatResponse >	Expand all	object
EpreuveRequest >	Expand all	object
EpreuveResponse >	Expand all	object
DisciplineRequest >	Expand all	object
DisciplineResponse >	Expand all	object
AthleteRequest >	Expand all	object
AthleteResponse >	Expand all	object
TableauMedailleResponse >	Expand all	object
ClassementPaysResponse >	Expand all	object
DashboardResponse >	Expand all	object
MedaillePaysResponse >	Expand all	object
MedailleStatResponse >	Expand all	object

Exemple de résultat attendu

```
{
  "nombreTotalAthletes": 9,
  "nombrePaysParticipants": 4,
  "medailles": {
    "nombreOr": 3,
    "nombreArgent": 3,
    "nombreBronze": 3,
    "total": 9
  }
}
```

6. Service SOAP

Le service **SOAP** utilise une approche contract-first. Le contrat est d'abord défini dans le fichier **olympics.xsd**. Maven génère ensuite les classes JAXB utilisées par l'endpoint Spring-WS.

➤ Adresses du service

- WSDL : <http://localhost:8081/ws/olympics.wsdl>
- Endpoint : POST <http://localhost:8081/ws/>
- Opération : `getTableauMedailles`

➤ Chaîne de traitement

Requête XML → MessageDispatcherServlet → TableauMedailleEndpoint → TableauMedailleService → ResultatRepository → réponse JAXB → enveloppe SOAP.

Exemple de requête

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:oly="http://olympics.com/management/soap">
  <soapenv:Header/>
  <soapenv:Body>
    <oly:getTableauMedaillesRequest/>
  </soapenv:Body>
</soapenv:Envelope>
```

Capture du SOAP dans Postman avec le résultat 200 OK :



7. Tests, Démonstration Et Résultats

La validation du projet repose sur la compilation **Maven**, les tests manuels **REST** dans **Swagger** et **Postman**, le contrôle du WSDL, le test SOAP et la vérification des données MySQL.

Test	Outil	Résultat attendu	Statut
Compilation	Maven	BUILD SUCCESS	Validé
CRUD Discipline	Swagger/Postman	Codes 200/201/204	Validé
Email en doublon	Swagger/Postman	Erreur métier 400	Validé
Résultat incompatible	Swagger/Postman	Erreur métier 400	Validé
Tableau des médailles REST	Swagger	Réponse JSON classée	Validé
Dashboard	Swagger	Statistiques cohérentes	Validé
WSDL	Navigateur	Document XML affiché	Validé
SOAP	Postman	200 OK + réponse XML	Validé

➤ Collection Postman

La collection fournie regroupe les requêtes **REST** et **SOAP**. Après importation, la variable baseUrl est définie à <http://localhost:8081>. Les dossiers sont organisés par module pour faciliter la démonstration.

8. Difficultés Rencontrées Et Solutions

Difficulté	Solution appliquée
Méthodes repository supprimées lors d'une modification	Restauration de existsByEmailIgnoreCase et findByEmailIgnoreCase.
Dépendance SOAP déclarée deux fois	Suppression du doublon dans pom.xml.
Confusion entre /ws et une page web	Utilisation de GET pour le WSDL et POST pour l'endpoint SOAP.
Validation des relations métier	Ajout de BusinessException et contrôles dans le service.
Génération des classes SOAP	Configuration du plugin JAXB2 Maven à partir du XSD.

9. Conclusion Et Perspectives

Olympic Management System répond aux objectifs fonctionnels et techniques définis. L'application propose une architecture structurée, une persistance MySQL, des API REST documentées, un service SOAP opérationnel et un tableau de bord statistique. Les règles métier importantes sont contrôlées et les erreurs sont retournées sous une forme uniforme.

PERSPECTIVES D'EVOLUTION

- ✚ Ajouter l'authentification et les rôles avec Spring Security et JWT.
- ✚ Créer une interface web ou mobile consommant les API.
- ✚ Ajouter des tests unitaires et d'intégration automatisés.
- ✚ Conteneuriser l'application et MySQL avec Docker.
- ✚ Ajouter la pagination, le filtrage et l'export PDF/Excel.
- ✚ Mettre en place une intégration continue.

FIN